CE 222 – *Computer Organization and Assembly Language (COAL)*
Lecture 14

Dr. Asad Mahmood

Feb. 17, 2026

Lecture Outline

- Announcements (Assignment and Quiz 2 Next week – Chapter 2)
- Outline of Previous Lecture:
  - **Chapter 2 - Instructions: Language of the Computer**
    - 2.8 Supporting Procedures in Computer Hardware

- Outline to Today's Lecture:
  - **Chapter 2 - Instructions: Language of the Computer**
    - 2.8 Supporting Procedures in Computer Hardware
    - 2.9 onwards – Mostly Reading

Chapter 2

Instructions: Language of the Computer

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

Procedures

- Procedures in assembly language is one tool programmers use to structure programs for
  - Ease of understanding
  - Code re-reuse
- Similar to *functions/methods* in high level languages like C, Python
- Parameters act as an interface between the procedure and the rest of the program

- Spy analogy – cover the tracks!

Procedure Calling

- Following steps are followed when calling procedures:
  1. Place parameters in registers (x10 to x17)
  2. Transfer control to procedure / callee
  3. Acquire storage for procedure
  4. Perform procedure's operations
  5. Place result in register (x10 to x17) for caller
  6. Return to place of call (address in x1)

Procedure Call Instructions

- Some dedicated RISC-V instructions for procedures
- To call a procedure: jump and link (`jal`)

  `jal x1, ProcedureLabel`
  - Address of following instruction put in x1
  - Jumps to target address
- To return from a Procedure: jump and link register (`jalr`)

  `jalr x0, 0(x1)`
  - Return address?
- Program counter (PC): A register to store the address of current instruction being executed
  - Value in x1 wrt PC

Using more Registers

- If a compiler needs more registers for a procedure than the eight argument registers, portion of memory/RAM used – termed spilling to the memory!
- A special portion of memory, known as Stack, is used for such purpose:
  - Stack works as a last-in-first-out queue.
  - Stack Pointer (register x2)
  - Push – place data on stack
  - Pop – Remove data from stack
  - Grows downwards!

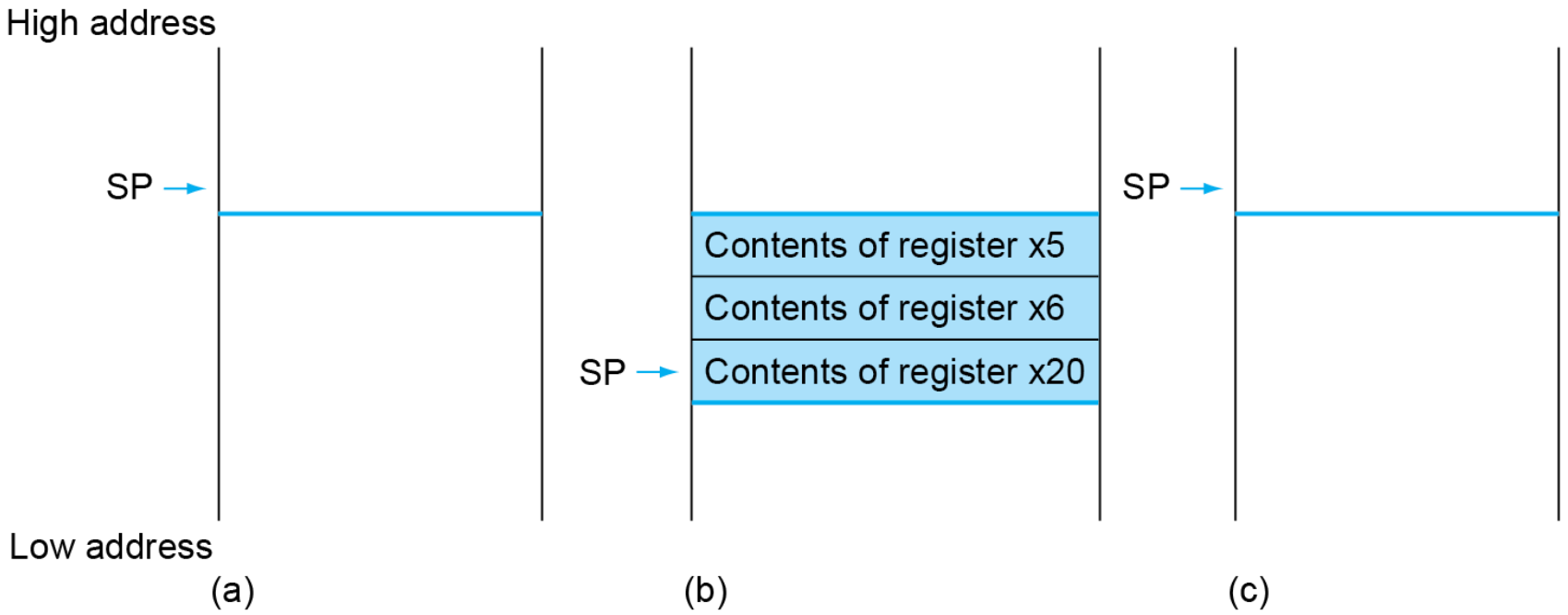
Leaf Procedure Example

- C code:

```
int leaf_example (int g, int h, int i, int j)
{
  int f;
  f = (g + h) - (i + j);
  return f;
}
```

  - Arguments g, …, j in x10, …, x13
  - f in x20
  - temporaries x5, x6
  - Need to save x5, x6, x20 on stack
- RISC-V Code

Local Data on the Stack



Register Usage

- x5 – x7, x28 – x31: temporary registers
  - Not preserved by the callee
- x8 – x9, x18 – x27: saved registers
  - If used, the callee saves and restores them

Non-Leaf Procedures

- Procedures that call other procedures
- For nested/recursive calls, a conflict in parameters and return address registers can occur -> avoided via use of stack!
- For nested call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call
  - Stack pointer (sp) updated accordingly!
- Restore from the stack after the call

Non-Leaf Procedure Example

- C code:

```
int fact (int n)
{
  if (n < 1)
        return 1;
  else
        return n * fact(n - 1);
}
```

  - Argument n in x10
  - Result in x10

Non-Leaf Procedure Example

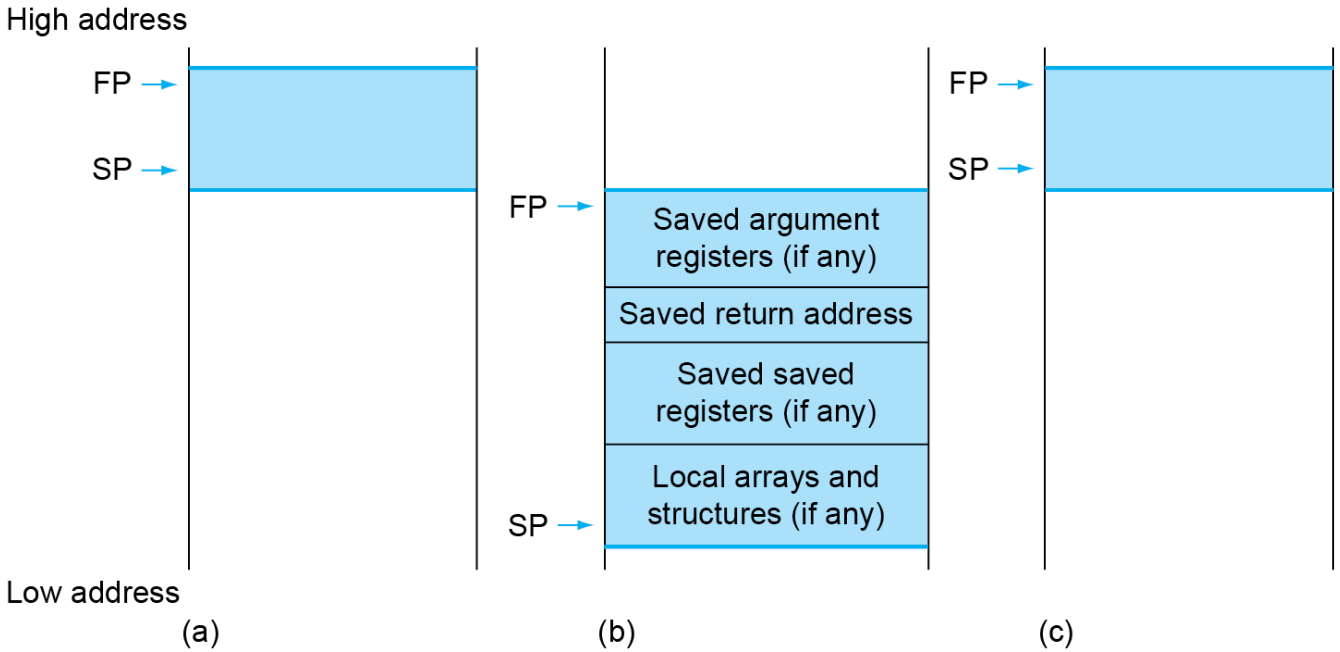
- RISC-V code:

```
fact:
    1 addi sp,sp,-8
    2 sw   x1,4(sp)
    3 sw   x10,0(sp)
    4 addi x5,x10,-1
    5 bge  x5,x0,L1
    6 addi x10,x0,1
    7 addi sp,sp,8
    8 jalr x0,0(x1)
9 L1: addi x10,x10,-1
   10 jal  x1,fact
   11 addi x6,x10,0
   12 lw   x10,0(sp)
   13 lw   x1,4(sp)
   14 addi sp,sp,8
   15 mul  x10,x10,x6
   16 jalr x0,0(x1)
```

Register and Memory Preservation

Preserved	Not preserved
Saved registers: x8-x9, x18-x27	Temporary registers: x5-x7, x28-x31
Stack pointer register: x2(sp)	Argument/result registers: x10-x17
Frame pointer: x8(fp)	
Return address: x1(ra)	
Stack above the stack pointer	Stack below the stack pointer

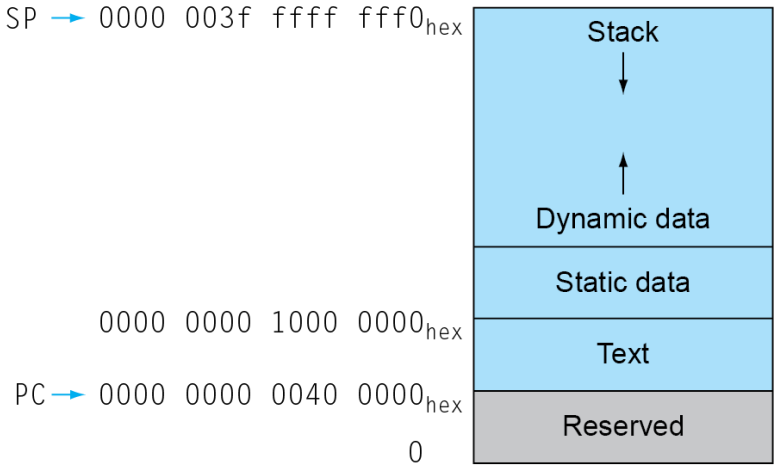
Local Data on the Stack



- Local data allocated by callee
  - e.g., C automatic variables
- Procedure frame (activation record)
  - Used by some compilers to manage stack storage

Memory Layout

- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - x3 (global pointer) initialized to address allowing ±offsets into this segment
- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage



Tail Recursions

- Efficient Implementation possible

- Example (C Code)

```
int sum (int n, int acc)
{
  if (n > 0)
       return sum(n – 1, acc + n);
  else
       return acc;
}
```

  - x10 =n, x11 = acc, x12 = result/return value

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

Character Data

- Byte-encoded character sets
  - ASCII: 128 characters
    - 95 graphic, 33 control
  - Latin-1: 256 characters
    - ASCII, +96 more graphic characters
- Unicode: 32-bit character set
  - Used in Java, C++ wide characters, …
  - Most of the world's alphabets, plus symbols
  - UTF-8, UTF-16: variable-length encodings

Byte/Halfword/Word Operations

- RISC-V byte/halfword/word load/store
  - Load byte/halfword/word: Sign extend to 32 bits in rd
    - `lb rd, offset(rs1)`
    - `lh rd, offset(rs1)`
    - `lw rd, offset(rs1)`
  - Load byte/halfword/word unsigned: Zero extend to 32 bits in rd
    - `lbu rd, offset(rs1)`
    - `lhu rd, offset(rs1)`
    - `lwu rd, offset(rs1)`
  - Store byte/halfword/word: Store rightmost 8/16/32 bits
    - `sb rs2, offset(rs1)`
    - `sh rs2, offset(rs1)`
    - `sw rs2, offset(rs1)`

String Copy Example

- C code:
  - Null-terminated string

```
void strcpy (char x[], char y[])
{    size_t i;
     i = 0;
     while ((x[i]=y[i])!='\0')
     i += 1;
}
```

- Assume base addresses of x[] and y[] in x10 and 11, respectively. ‘i’ in x19

String Copy Example

- RISC-V code:

```
strcpy:
        addi sp,sp,-4          // adjust stack for 1 word
        sw   x19,0(sp)         // push x19 to stack
        add  x19,x0,x0         // i=0

L1:  add  x5,x19,x10           // x5 = addr of y[i]
        lbu  x6,0(x5)          // x6 = y[i]
        add  x7,x19,x10        // x7 = addr of x[i]
        sb   x6,0(x7)          // x[i] = y[i]
        beq  x6,x0,L2          // if y[i] == 0 then exit
        addi x19,x19,1         // i = i + 1
        jal  x0,L1             // next iteration of loop

L2:  lw   x19,0(sp)            // restore saved x19
        addi sp,sp,4           // pop 1 word from stack
        jalr x0,0(x1)          // and return
```

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

32-bit Constants

- Most constants are small
  - 12-bit immediate is sufficient
- For the occasional 32-bit constant

  `lui rd, constant` (U-type!)
  - To copy a 32 bit constant, first its left-most 20 bits are copied to bits [31:12] of rd using `lui`
  - Then the rest 12 bits are added
  - Example


```
lui x19, 976  // 0x003D0
```

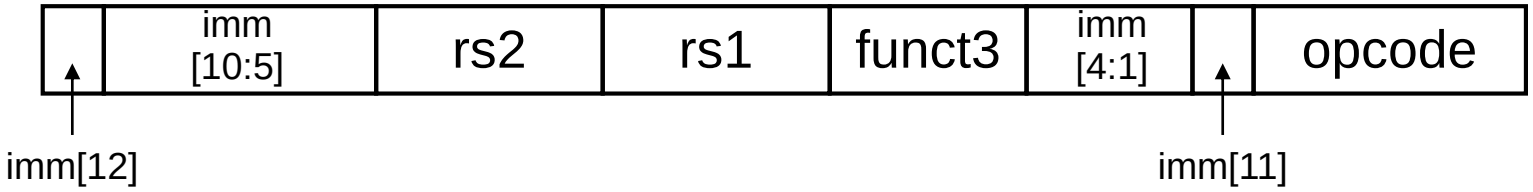
0000 0000 0011 1101 0000	0000 0000 0000

```
addi x19,x19,1280  // 0x500
```

0000 0000 0011 1101 0000	0101 0000 0000

Branch Addressing

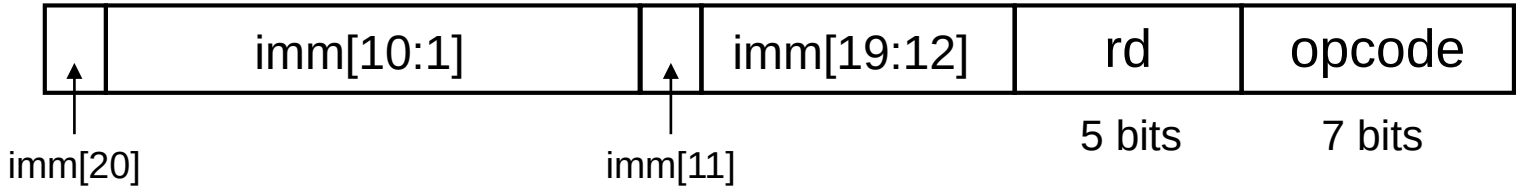
- Branch instructions specify
  - Opcode, two registers, target address
- Most branch targets are near branch
  - Forward or backward
- SB format:



- PC-relative addressing
  - Target address = PC + immediate × 2

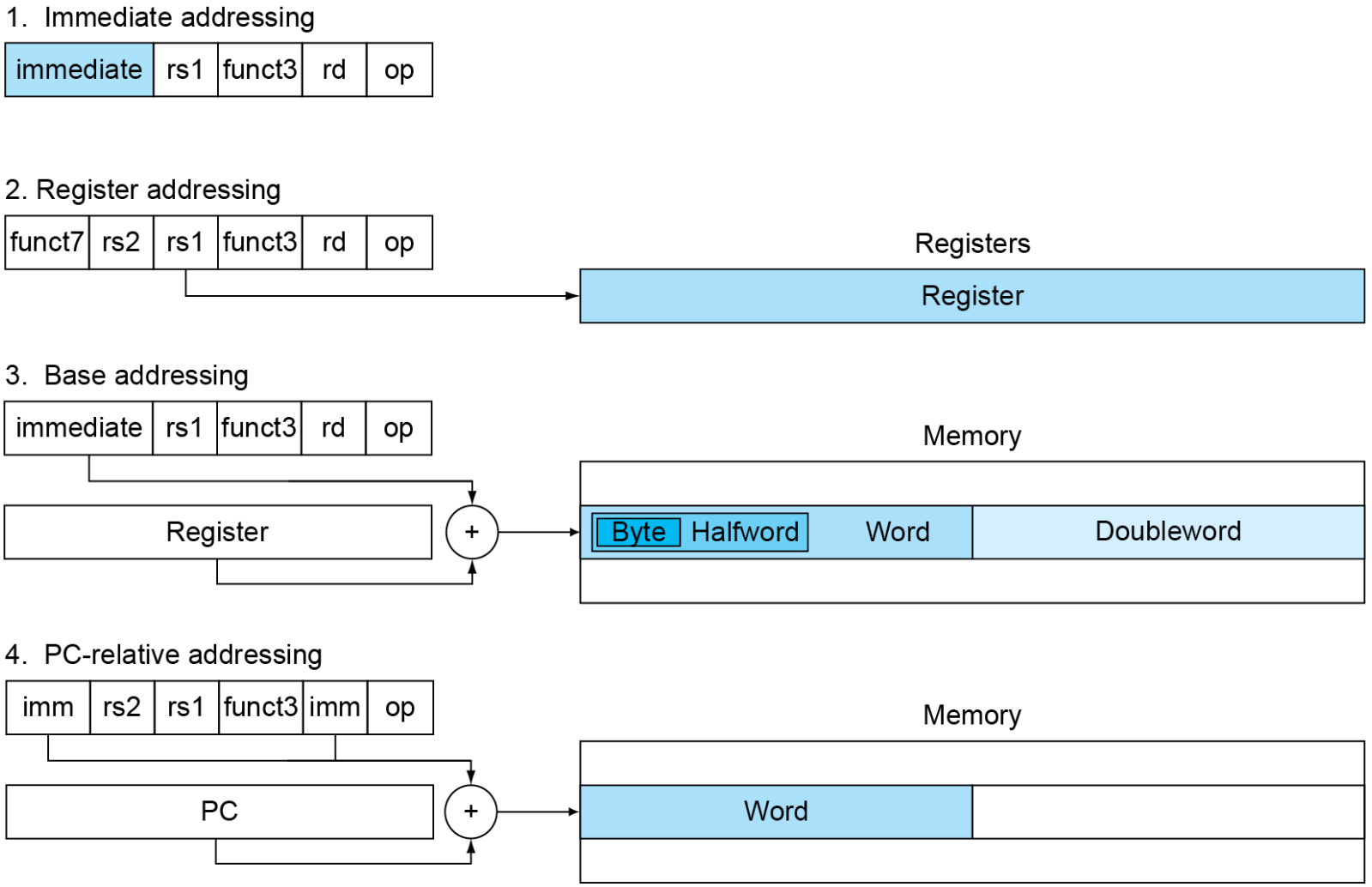
Jump Addressing

- Jump and link (`jal`) target uses 20-bit immediate for larger range
- UJ format:



- For long jumps, eg, to 32-bit absolute address
  - lui: load address[31:12] to temp register
  - jalr: add address[11:0] and jump to target

RISC-V Addressing Summary



RISC-V Encoding Summary

Name (Field Size)	Field 7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

Synchronization

- Two processors sharing an area of memory
  - P1 writes, then P2 reads
  - Data race if P1 and P2 don’t synchronize
- Hardware support required
  - Atomic read/write memory operation
  - No other access to the location allowed between the read and write
- Could be a single instruction
  - E.g., atomic swap of register ↔ memory
  - Or an atomic pair of instructions

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

Translation and Startup